AlgoFlow

PRODUCTION DSA PLATFORM

SQL Notes

From your first database to joins, subqueries and views. Examples use MySQL syntax.

MYSQL | CRUD | JOINS | INTERVIEW READY

1. Databases and SQL

A **database** is an organised, digital collection of data that can be stored, searched and updated efficiently. The software that manages it is a **DBMS** (Database Management System), e.g. MySQL, Oracle, SQL Server.

Type	How data is stored	Examples
Relational (RDBMS)	In tables (rows and columns) linked by keys; queried with SQL	MySQL, Oracle, SQL Server
Non-relational (NoSQL)	Not in tables: documents, key-value pairs, graphs, wide columns	MongoDB, Redis, Cassandra

SQL (Structured Query Language) is the standard language for talking to relational databases. Everything you do with data falls under **CRUD**: **C**reate, **R**ead, **U**pdate, **D**elete.

Structure: a database server holds many *databases*; each database holds many *tables*; each table holds *rows* (records) and *columns* (fields).

Sample tables used in these notes

employees

emp_id	name	city	salary	dept_id
1	Riya	Pune	62000	10
2	Kabir	Indore	48000	20
3	Meera	Pune	75000	10
4	Arjun	Delhi	39000	30
5	Sana	Indore	91000	NULL

departments

dept_id	dept_name
10	Engineering
20	Marketing
30	Support
40	Finance

Sana has no department; Finance has no employees. These two rows make the join results easy to tell apart.

2. Types of SQL commands

Category	Purpose	Commands
DDL – Data Definition	Define / change structure	CREATE, ALTER, RENAME, TRUNCATE, DROP
DQL – Data Query	Read data	SELECT
DML – Data Manipulation	Change the data in tables	INSERT, UPDATE, DELETE
DCL – Data Control	Permissions	GRANT, REVOKE
TCL – Transaction Control	Group changes into transactions	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

3. Database and table basics

```sql
CREATE DATABASE IF NOT EXISTS company;    -- IF NOT EXISTS avoids an error
SHOW DATABASES;
USE company;                               -- pick the database to work in
SHOW TABLES;
DROP DATABASE IF EXISTS company;           -- deletes everything inside!
```

Creating a table

```sql
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id  INT PRIMARY KEY AUTO_INCREMENT,
    name    VARCHAR(50) NOT NULL,
    city    VARCHAR(30) DEFAULT 'Indore',
    salary  INT CHECK (salary >= 0),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Inserting rows

```sql
INSERT INTO employees (name, city, salary, dept_id)
VALUES ('Riya', 'Pune', 62000, 10),
       ('Kabir', 'Indore', 48000, 20);       -- many rows in one statement

SELECT * FROM employees;                     -- view every column
```

4. Common datatypes (MySQL)

Datatype	Stores	Example
`INT` / `BIGINT`	Whole numbers (4 / 8 bytes)	emp_id INT
`TINYINT`	Small whole numbers, 1 byte	age TINYINT UNSIGNED
`DECIMAL(p, s)`	Exact decimals: p digits total, s after the point	price DECIMAL(8,2)
`FLOAT` / `DOUBLE`	Approximate decimals	rating FLOAT
`CHAR(n)`	Fixed-length text (always n chars)	gender CHAR(1)
`VARCHAR(n)`	Variable-length text, up to n chars	name VARCHAR(50)
`TEXT`	Long text	bio TEXT
`BOOLEAN`	True / false (stored as 1 / 0)	is_active BOOLEAN
`DATE` / `DATETIME`	Date (YYYY-MM-DD) / date + time	joined DATE

Signed vs unsigned: a signed column allows negatives; `UNSIGNED` shifts the whole range to non-negative. TINYINT holds −128 to 127, while TINYINT UNSIGNED holds 0 to 255. Use UNSIGNED for values that can never be negative (age, quantity).

5. Keys and constraints

	Primary key (PK)	Foreign key (FK)
Meaning	Column(s) that uniquely identify each row	Column(s) that point to the PK of another table
How many	Exactly one per table	Any number
NULL / duplicates	Never NULL, never duplicate	Both allowed
In our example	employees.emp_id	employees.dept_id → departments.dept_id

Constraint	Rule it enforces
`NOT NULL`	Column must always have a value
`UNIQUE`	No two rows can share a value (NULLs allowed)
`PRIMARY KEY`	NOT NULL + UNIQUE; one per table
`FOREIGN KEY`	Value must exist in the referenced table, so links can't break
`DEFAULT`	Value used when none is given on insert
`CHECK`	Value must satisfy a condition, e.g. `CHECK (salary >= 0)`

Cascading on foreign keys

```
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    ON DELETE CASCADE     -- delete a department → its employees' rows are deleted too
    ON UPDATE CASCADE     -- change a dept_id in the parent → child rows follow
```

Other options: `ON DELETE SET NULL` (child's FK becomes NULL) and the default `RESTRICT` (block the delete).

6. Reading data: SELECT in detail

```
SELECT name, salary FROM employees;        -- specific columns
SELECT DISTINCT city FROM employees;       -- unique values only
```

WHERE: filter rows

```
SELECT * FROM employees WHERE salary > 50000 AND city = 'Pune';
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 70000;  -- inclusive
SELECT * FROM employees WHERE city IN ('Delhi', 'Indore');
SELECT * FROM employees WHERE city NOT IN ('Delhi');
SELECT * FROM employees WHERE name LIKE 'M%';     -- starts with M; _ = one char
SELECT * FROM employees WHERE dept_id IS NULL;    -- never use = NULL
```

Operator group	Operators
Arithmetic	+ , − , * , / , %
Comparison	= , != (or <>) , > , >= , < , <=
Logical	AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL, ANY, ALL
Bitwise	& (AND), | (OR)

ORDER BY and LIMIT

```
SELECT name, salary FROM employees
ORDER BY salary DESC          -- ASC is the default
LIMIT 3;                       -- top 3 earners
```

7. Aggregates, GROUP BY and HAVING

Aggregate functions take many values and return one: `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.

```
SELECT AVG(salary) FROM employees;              -- 63000
SELECT MAX(salary), COUNT(*) FROM employees;    -- 91000, 5
```

GROUP BY collapses rows with the same value into one summary row, almost always with an aggregate:

```
SELECT city, COUNT(*) AS total, AVG(salary) AS avg_salary
FROM employees
GROUP BY city;
```

HAVING filters *groups*, the way WHERE filters *rows*. It runs after grouping, so it can use aggregates:

```
SELECT city, COUNT(*) FROM employees
GROUP BY city
HAVING MAX(salary) > 70000;      -- Pune and Indore only
```

Clause order (write it in this order):
`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`
Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That's why you can't use an aggregate in WHERE.

8. Changing data and structure

UPDATE and DELETE rows

```
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 10;
DELETE FROM employees WHERE salary < 40000;
```

Always write the **WHERE** clause with UPDATE and DELETE. Without it, every row changes. MySQL Workbench's safe-update mode blocks this; turn it off with `SET SQL_SAFE_UPDATES = 0;` only when you mean it.

ALTER: change the schema

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100) UNIQUE;
ALTER TABLE employees DROP COLUMN email;
ALTER TABLE employees MODIFY name VARCHAR(80) NOT NULL;          -- new type/constraint
ALTER TABLE employees CHANGE COLUMN name full_name VARCHAR(80);  -- rename + type
ALTER TABLE employees RENAME TO staff;
```

DELETE vs TRUNCATE vs DROP

Command	Removes	WHERE?	Table remains?
`DELETE FROM t`	Chosen rows (or all)	Yes	Yes
`TRUNCATE TABLE t`	All rows, fast; resets AUTO_INCREMENT	No	Yes
`DROP TABLE t`	Rows and the table itself	No	No

9. Joins

A join combines rows from two or more tables using a related column. All examples join on `e.dept_id = d.dept_id`.

Join	Returns	Rows from our sample
INNER JOIN	Only rows with a match in both tables	Riya, Kabir, Meera, Arjun
LEFT JOIN	All left rows + matches (NULL if none)	All 5 employees; Sana's dept is NULL
RIGHT JOIN	All right rows + matches (NULL if none)	4 matched rows + Finance with NULL employee
FULL JOIN	All rows from both sides	All employees + Finance

```sql
-- INNER
SELECT e.name, d.dept_name
FROM employees AS e
INNER JOIN departments AS d ON e.dept_id = d.dept_id;

-- LEFT / RIGHT: swap the keyword
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

FULL JOIN in MySQL (no FULL OUTER JOIN keyword)

```sql
SELECT * FROM employees e LEFT JOIN  departments d ON e.dept_id = d.dept_id
UNION
SELECT * FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Exclusive joins (rows with no match)

```sql
-- Left exclusive: employees with no department → Sana
SELECT e.* FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;

-- Right exclusive: departments with no employees → Finance
SELECT d.* FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

Self join

A table joined with itself, using two aliases. Classic case: an employee table with a `manager_id` column that points to another employee's `emp_id`.

```sql
SELECT e.name AS employee, m.name AS manager
FROM staff AS e
JOIN staff AS m ON e.manager_id = m.emp_id;
```

10. UNION

Stacks the results of two SELECTs into one result. `UNION` removes duplicates; `UNION ALL` keeps them (and is faster).

- Both SELECTs must return the same number of columns.
- Matching columns must have compatible datatypes and be in the same order.

```
SELECT city FROM employees
UNION
SELECT city FROM customers;
```

11. Subqueries

A subquery (inner or nested query) is a SELECT inside another query. The inner query runs first and its result feeds the outer one. It can appear in **WHERE**, **FROM** or **SELECT**.

```
-- In WHERE: employees earning more than the company average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- With IN: employees with an even emp_id
SELECT name FROM employees
WHERE emp_id IN (SELECT emp_id FROM employees WHERE emp_id % 2 = 0);

-- In FROM (derived table needs an alias): top salary among Pune employees
SELECT MAX(salary)
FROM (SELECT * FROM employees WHERE city = 'Pune') AS pune_emp;
```

12. Views

A **view** is a saved query that behaves like a virtual table. It stores no data of its own; it is re-run each time you query it, so it always shows current data. Use it to simplify complex queries or to expose only some columns.

```
CREATE VIEW emp_public AS
SELECT name, city FROM employees;     -- hides salary

SELECT * FROM emp_public;
DROP VIEW emp_public;
```

Quick revision

1. WHERE filters rows before grouping; HAVING filters groups after.
2. PK = unique + not null, one per table. FK links to another table's PK; can repeat or be NULL.
3. Use `IS NULL`, never `= NULL`.
4. DELETE removes rows (can filter), TRUNCATE empties the table, DROP removes the table.
5. INNER = matches only; LEFT/RIGHT = all of one side; FULL in MySQL = LEFT UNION RIGHT.
6. Exclusive join = outer join + `WHERE other_side.key IS NULL`.
7. UNION removes duplicates; UNION ALL keeps them.